



UNIVERSITÉ CHEIKH AHMADOUL KHADIM

Rapport de TP : GraphQL & Microservices

Réalisé par l'équipe :

Makane Seck

Mor Faye

Mame Amy Lo

Makhtar Wade

Sommaire

1. Architecture du Système

2. Procédure de Déploiement Local

3. Validation Fonctionnelle (Tests)

- 3.1. Authentification de l'Administrateur

- 3.2. Configuration de l'En-tête d'Autorisation

- 3.3. Récupération de la Liste des Étudiants

- 3.4. Ajout d'un Nouvel Étudiant

- 3.5. Mise à Jour d'un Étudiant

- 3.6. Recherche par Paramètre (Variables GraphQL)

- 3.7. Suppression d'un Étudiant

4. Conclusion : Justification Technique de l'Architecture

Rapport de TP : Découverte de GraphQL et Introduction aux Microservices

1. Architecture du Système

Le projet repose sur une architecture orientée microservices composée de trois entités distinctes :

1. **Service d'Authentification (Port 3001)** : Microservice en charge de la validation des identifiants administrateur et de la génération des tokens JWT pour la sécurisation des échanges.
 2. **Service des Étudiants (Port 3002)** : Microservice métier responsable des opérations CRUD (Création, Lecture, Mise à jour, Suppression) sur l'entité Étudiant.
 3. **Passerelle (Gateway) GraphQL (Port 4000)** : Point d'entrée unique de l'architecture. Elle orchestre les requêtes, vérifie les autorisations d'accès (validation du token JWT) et relaie les requêtes vers le Service des Étudiants.
-

2. Procédure de Déploiement Local

L'initialisation de l'application nécessite l'exécution simultanée des trois services. Il est recommandé d'utiliser des terminaux distincts.

Service d'Authentification :

```
cd "ucak-microservices-graphql/auth-service"
node index.js
```

Service des Étudiants :

```
cd "ucak-microservices-graphql/student-service"
node index.js
```

Passerelle GraphQL :

```
cd "ucak-microservices-graphql/graphql-gateway"
node index.js
```

The image shows three terminal windows from a macOS environment, demonstrating the setup and execution of a GraphQL microservices architecture.

Terminal 1 (Top): The user is in the directory `~/Documents/Veille Technologie/ucak-microservices-graphql/student-service`. They run `node index.js`, and the output is "Student Service is running on http://localhost:3002".

Terminal 2 (Middle): The user is in the directory `~/Documents/Veille Technologie/ucak-microservices-graphql`. They run a series of commands: `ls` (listing files like `auth-service`, `graphql-gateway`, `student-service`), `cd graphql-gateway`, `cd ..`, `cd auth-service`, and `node index.js`. The output is "Auth Service is running on http://localhost:3001".

Terminal 3 (Bottom): The user is in the directory `~/Documents/Veille Technologie/ucak-microservices-graphql/graphql-gateway`. They run `ls` (listing files like `index.js`, `package-lock.json`, `package.json`, `resolvers.js`, `schema.js`), and then `node index.js`. The output is "GraphQL Gateway is running on http://localhost:4000/graphql".

3. Validation Fonctionnelle (Tests)

Les tests suivants permettent de valider le bon fonctionnement de l'API GraphQL via l'interface Apollo Sandbox, accessible à l'adresse :

`http://localhost:4000/graphql`.`

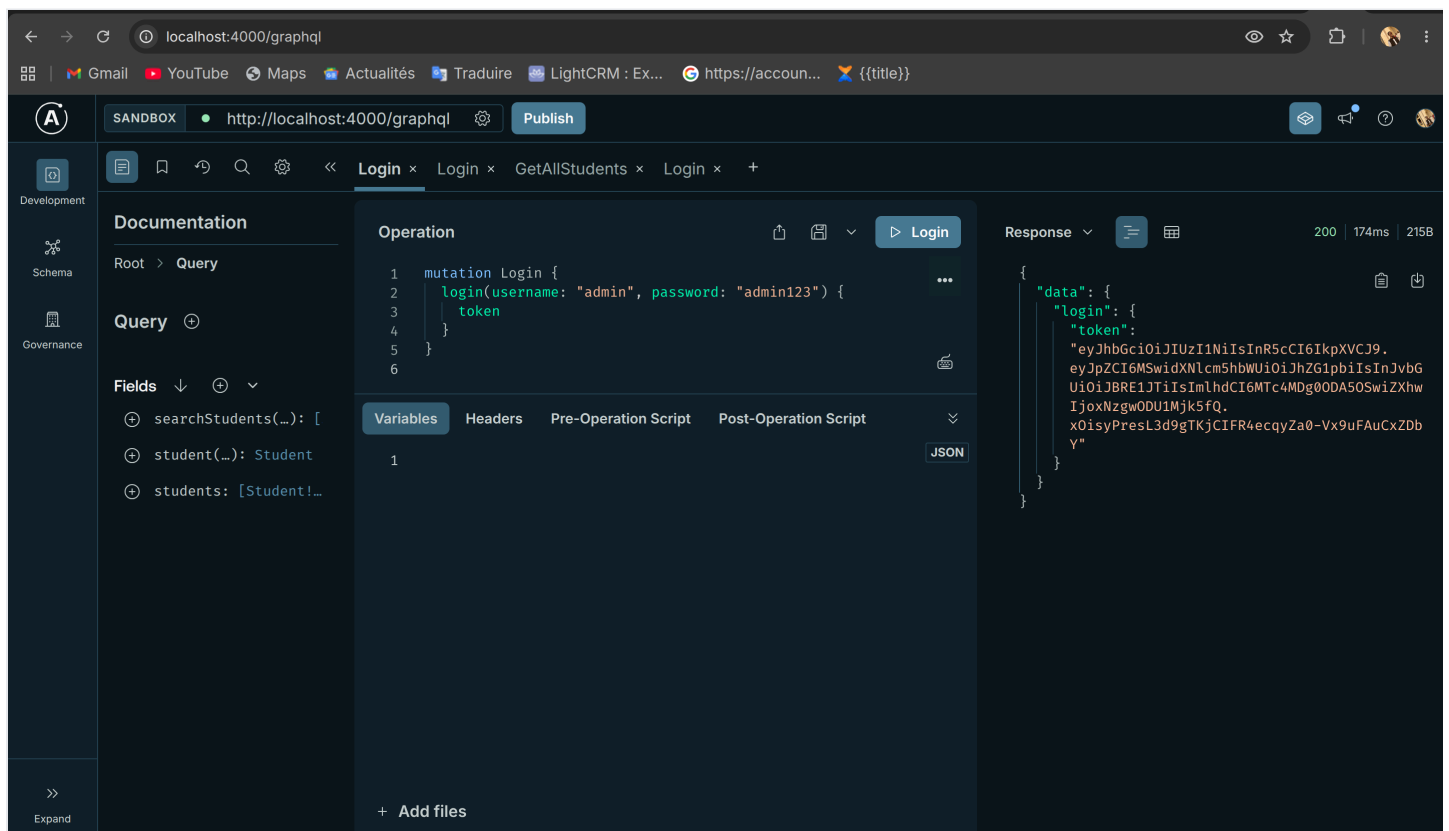
3.1. Authentification de l'Administrateur

L'authentification est un prérequis pour l'exécution des requêtes protégées.

Requête (Mutation) :

```
mutation Login {  
  login(username: "admin", password: "admin123") {  
    token  
  }  
}
```

Résultat attendu : Retourne un token JWT valide.



3.2. Configuration de l'En-tête d'Autorisation

Pour autoriser les appels suivants, le token récupéré doit être inséré dans les en-têtes HTTP de la requête client :

- Clé : ``Authorization``
- Valeur : ``Bearer <VOTRE_TOKEN>``

3.3. Récupération de la Liste des Étudiants

Requête (Query) :

```
query GetAllStudents {  
  students {  
    id  
    prenom  
    nom  
    email  
    matricule  
    filiere  
    niveau  
  }  
}
```

Résultat attendu : Retourne le tableau contenant l'ensemble des étudiants.

The screenshot shows the GraphQL Playground interface. The query editor contains the following query:

```
1 query GetAllStudents {
2   students {
3     id
4     prenom
5     nom
6     email
7     matricule
8     filiere
9     niveau
10  }
11 }
12
```

The response is displayed in a table format:

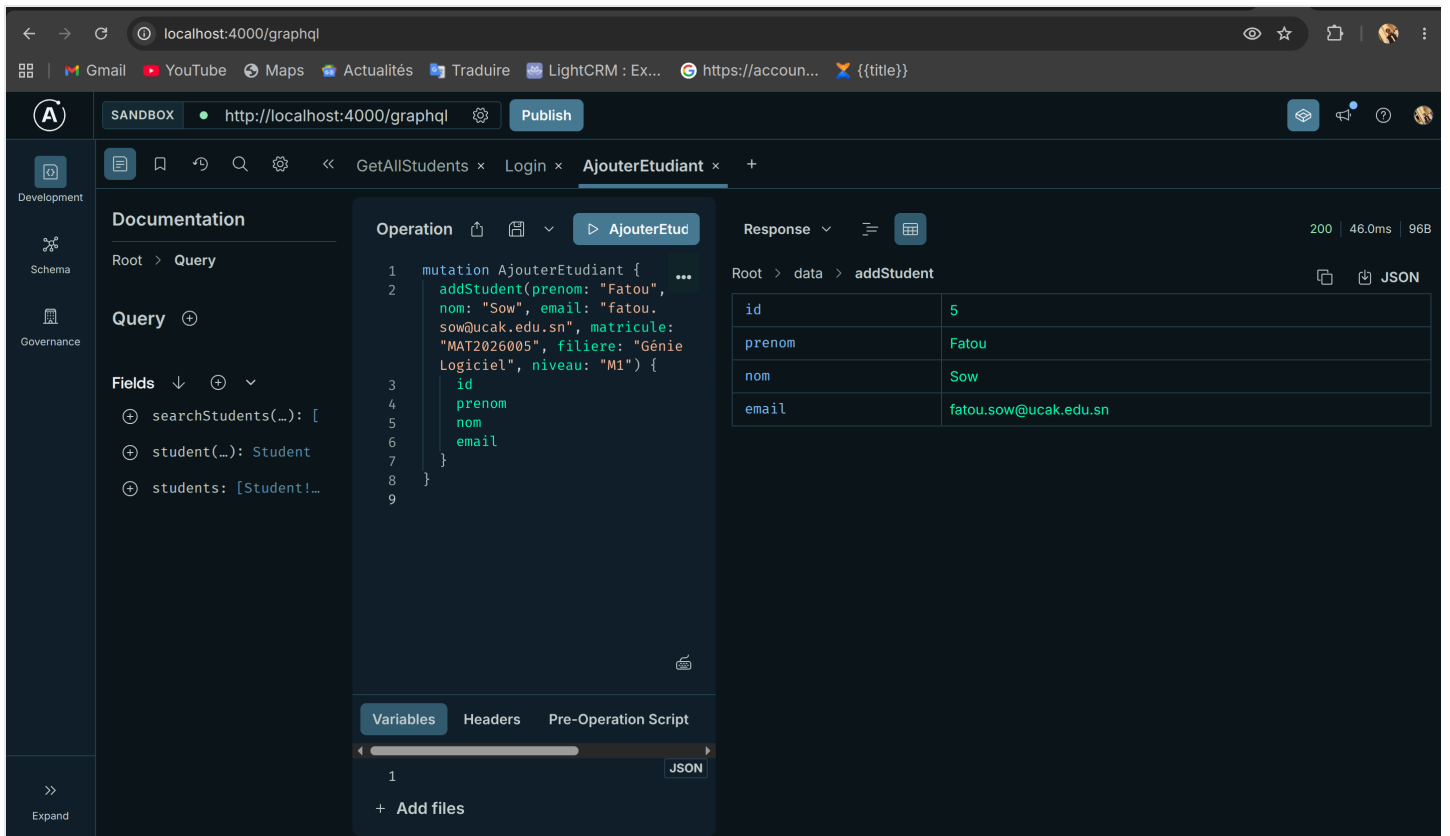
id	prenom	nom	email	matricule
0	1	Makhtar Wade	makhtar.wade@ucak.edu.sn	MAT2026001
1	3	Makane Seck	makane.seck@ucak.edu.sn	MAT2026003
2	4	Amy Lo	amy.lo@ucak.edu.sn	MAT2026004

3.4. Ajout d'un Nouvel Étudiant

Requête (Mutation) :

```
mutation AjouterEtudiant {
  addStudent(prenom: "Fatou", nom: "Sow", email: "fatou.sow@ucak.edu.sn", matricule: "MAT2026002") {
    id
    prenom
    nom
    email
  }
}
```

Résultat attendu : Confirme la création de l'étudiant avec l'ID correspondant.



3.5. Mise à Jour d'un Étudiant

Requête (Mutation) :

```
mutation ModifierEtudiant {  
  updateStudent(id: "5", prenom: "Fatou", nom: "Sow", email: "fatou.nouvel_email@ucak.edu.sn") {  
    id  
    email  
  }  
}
```

Résultat attendu : Confirme la modification des champs spécifiés.

3.6. Recherche par Paramètre (Variables GraphQL)

Requête (Query) :


```
query ChercherEtudiant($texteRecherche: String!) {  
  searchStudents(q: $texteRecherche) {  
    id  
    prenom  
    nom  
    email  
  }  
}
```

Variables :

```
{  
  "texteRecherche": "Makane"  
}
```

Résultat attendu : Retourne uniquement les entités correspondant à la chaîne de caractères.

3.7. Suppression d'un Étudiant

Requête (Mutation) :

```
mutation SupprimerEtudiant {  
  deleteStudent(id: "5") {  
    id  
    prenom  
    nom  
  }  
}
```

Résultat attendu : Confirme la suppression et retourne les informations de l'étudiant supprimé.

4. Conclusion : Justification Technique de l'Architecture

- **Optimisation de la bande passante (GraphQL)** : GraphQL élimine les problématiques d'Over-fetching et d'Under-fetching en permettant au client de requêter avec précision la structure des données désirée.
- **Point d'entrée centralisé** : L'API expose un unique endpoint (`/graphql`), masquant ainsi la complexité du réseau sous-jacent à l'application cliente.
- **Résilience et Scalabilité (Microservices)** : La séparation physique des composants (Auth et Student) isole les pannes éventuelles et offre la possibilité de mettre à l'échelle uniquement le service en surcharge (par exemple, le Student Service lors des périodes d'inscription massives).